Capgemini – The – Hartford Challenge: Optimization of Premium Investment Portfolios

Luke Sebold, Binayek Tiwari, and Iraj Nelson

YQuantum Hackathon 2026

Supported by QuEra, Capgemini Quantum Lab, and The Hartford

Abstract

Modern portfolio optimization frequently encounters computationally intractable combinatorial constraints, such as cardinality limits on asset selection. In response to the Capgemini and The Hartford Challenge at YQuantum 2026, we present a quantum computational approach to optimizing premium investment portfolios. We reformulate the classical Markowitz mean-variance objective with a discrete budget constraint into a Quadratic Unconstrained Binary Optimization (QUBO) problem, which is then mapped to an Ising Hamiltonian. To evaluate our model while circumventing the exponential memory overhead of classical state-vector simulation, we implement a Pauli Propagation technique alongside the Quantum Approximate Optimization Algorithm (QAOA). Applied to a reduced eight-asset universe with a four-asset budget, our QAOA simulation successfully identifies valid, high-quality asset subsets. By tracking the evolution of sparse observables rather than the full state vector, we demonstrate a scalable pathway for simulating and validating quantum algorithms in financial optimization.

1 Introduction

Modern financial portfolio management relies heavily on the foundational mean-variance framework introduced by Harry Markowitz [1]. The standard model seeks to balance expected returns against portfolio variance, mathematically capturing the inherent trade-off between risk and reward. While the continuous relaxation of this problem can be solved efficiently via convex quadratic programming, real-world investment strategies—such as those managing premium portfolios for institutions like The Hartford—often require strict combinatorial constraints. For instance, a portfolio manager may need to select a fixed number of assets to minimize transaction costs and maintain manageable oversight. Introducing discrete budget or cardinality constraints transforms the optimization into an

NP-hard problem, challenging classical computational limits as the asset universe expands.

Quantum computing has emerged as a novel paradigm for tackling this combinatorial complexity. By mapping financial optimization problems to physics-based energy landscapes, quantum algorithms can theoretically explore vast solution spaces more efficiently. The Quantum Approximate Optimization Algorithm (QAOA) is particularly well-suited for solving these problems on Noisy Intermediate-Scale Quantum (NISQ) devices. However, testing and benchmarking QAOA circuits for large asset universes is currently bottlenecked by classical simulation limits; tracking the full quantum state requires memory that scales exponentially with the number of qubits.

In this paper, developed for the Capgemini and The Hartford Challenge at the YQuantum 2026 hackathon, we present an end-to-end workflow for quantum portfolio optimization [4]. First, we detail the mathematical derivation for transforming the constrained Markowitz problem into a QUBO model, and subsequently into an Ising Hamiltonian suitable for quantum hardware. To address the classical simulation bottleneck, we apply a Pauli Propagation approach that tracks the evolution of sparse observables rather than the full 2^n-dimensional state vector. Finally, we benchmark our quantum simulation against classical combinatorial baselines to evaluate its efficacy in identifying optimal risk-adjusted asset configurations.

2 Mathematical Formulation

In this section we derive, step by step, how the classical Markowitz mean–variance portfolio optimization problem is reformulated as a Quadratic Unconstrained Binary Optimization (QUBO) problem, and how the QUBO is subsequently mapped onto an Ising Hamiltonian suitable for execution on quantum hardware.

2.1 Markowitz Mean–Variance Objective

Consider a universe of n assets. Let μ_i denote the expected return of asset i, and let σ_{ij} denote the covariance between the returns of assets i and j. We have the following optimization [1]:

$$\min_{w} \quad q \sum_i \sum_j w_i w_j \sigma_{i,j} - \sum_i w_i \mu_i \quad \text{such that} \quad \sum_i w_i = 1 \tag{1}$$

where w_i denotes the proportion of total wealth invested in asset i, and $q > 0$ denotes the investor's risk-aversion coefficient. The condition $\sum_i w_i = 1$ ensures that the portfolio is fully invested.

2.2 QUBO Formulation with Budget Constraint

A portfolio must satisfy a budget constraint requiring that exactly B assets be selected:

$$\sum_{i=1}^{n} w_i \;=\; B\,.$$

Because a QUBO problem must be *unconstrained*, we enforce this requirement by adding a quadratic penalty term with Lagrange multiplier $\lambda > 0$. The full problem becomes [4]

$$\min_{w} \quad q \sum_{i,j} w_i \sigma_{ij} w_j - \sum_i \mu_i w_i + \lambda \left(\sum_i w_i - B \right)^2 \tag{2}$$

The penalty vanishes when the budget constraint is satisfied and grows quadratically with any violation. The multiplier λ must be chosen carefully: if λ is too small the optimiser may ignore the budget; if λ is too large the risk/return terms become negligible.

2.3 Derivation of the Hamiltonian Operator

Starting from the QUBO formulation,

$$\min_{w} \qquad q\sum_{i,j} w_i\sigma_{ij}w_j - \sum_i \mu_i w_i + \lambda\left(\sum_i w_i - B\right)^2 \tag{3}$$

we expand the penalty term:

$$\min_{w} \qquad q\sum_{i,j} w_i\sigma_{ij}w_j - \sum_i \mu_i w_i + \lambda\left(\left(\sum_i w_i\right)^2 - 2B\sum_i w_i + B^2\right). \tag{4}$$

Since $w_i \in \{0,1\}$, we use the binary property $w_i^2 = w_i$. Then,

$$\min_{w} \qquad q\sum_{i,j} w_i\sigma_{ij}w_j - \sum_i \mu_i w_i + \lambda\left(\sum_{i,j} w_i w_j - 2B\sum_i w_i + B^2\right). \tag{5}$$

Collecting quadratic and linear terms gives

$$\min_{w} \qquad \sum_{i,j} w_i w_j\left(q\sigma_{ij} + \lambda\right) + \sum_i w_i\left(-\mu_i - 2\lambda B\right) + \lambda B^2. \tag{6}$$

Separating the diagonal contribution from the double sum, we obtain

$$\min_{w} \qquad 2\sum_{i<j} w_i w_j\left(q\sigma_{ij} + \lambda\right) + \sum_i w_i\left(q\sigma_{ii} + \lambda - \mu_i - 2\lambda B\right) + \lambda B^2. \tag{7}$$

The constant term λB^2 does not affect the location of the minimum, so it may be ignored in the optimisation.

Connection to the Ising model. Comparing the reduced QUBO form with the standard quadratic binary form,

$$H(w) = \sum_{i<j} J_{ij} w_i w_j + \sum_i h_i w_i + \text{constant},$$

we identify

$$J_{ij} = 2\left(q\sigma_{ij} + \lambda\right), \qquad \forall i < j,$$

and

$$h_i = q\sigma_{ii} + \lambda - \mu_i - 2\lambda B.$$

Hence, the standard Ising Hamiltonian is written as

$$H(\sigma) = \sum_{i<j} J_{ij}\,\sigma_i\sigma_j + \sum_j h_j\,\sigma_j \tag{8}$$

$$= \sigma^T J\sigma + h^T\sigma \tag{9}$$

where $\sigma_j \in \{-1, 1\}$ are spin variables.

2.4 From Hamiltonian to Unitary Operator

The standard Ising Hamiltonian for an n-qubit system is written as [3]

$$H = \sum_{i<j} J_{ij}\,Z_iZ_j + \sum_i h_i\,Z_i \tag{10}$$

where Z_i denotes the Pauli-Z operator acting on the i-th qubit, and J_{ij} and h_i are the coupling and local field coefficients, respectively.

The single-qubit Pauli-Z matrix is

$$Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{11}$$

For an n-qubit system, Z_i means that Z acts on qubit i and the identity acts on all other qubits. Hence,

$$Z_i = I^{\otimes(i-1)} \otimes Z \otimes I^{\otimes(n-i)}. \tag{12}$$

Similarly, for $i < j$, the interaction term Z_iZ_j is represented by placing Z on the i-th and j-th qubits and identity on all others [2] :

$$Z_iZ_j = I^{\otimes(i-1)} \otimes Z \otimes I^{\otimes(j-i-1)} \otimes Z \otimes I^{\otimes(n-j)}. \tag{13}$$

Thus, the Hamiltonian in (10) is an operator on the full 2^n-dimensional Hilbert space of the n-qubit system. Since the Pauli-Z matrix is Hermitian and diagonal, each operator Z_i and Z_iZ_j is also Hermitian and diagonal. Therefore, the Hamiltonian H, being a real linear combination of these terms, is itself Hermitian and diagonal.

Each diagonal entry of H corresponds to one computational basis state. In an n-qubit system, the computational basis is given by states of the form

$$|x_1, x_2, \ldots, x_n\rangle, \qquad x_i \in \{0, 1\}. \tag{14}$$

Since

$$Z|0\rangle = |0\rangle, \tag{15}$$
$$Z|1\rangle = -|1\rangle, \tag{16}$$

the action of Z_i on the basis state $|x_1, x_2, \ldots, x_n\rangle$ gives the eigenvalue

$$s_i = 1 - 2x_i, \tag{17}$$

so that $s_i = +1$ when $x_i = 0$ and $s_i = -1$ when $x_i = 1$.

Hence, for any computational basis state $|x_1, x_2, \ldots, x_n\rangle$, the corresponding diagonal entry of H is

$$E(x_1, \ldots, x_n) = \sum_{i<j} J_{ij}\, s_i s_j + \sum_i h_i\, s_i. \tag{18}$$

Therefore, the unitary operator

$$U(\gamma) = e^{-i\gamma H} \tag{19}$$

is also diagonal in the computational basis. For any basis state indexed by k, its corresponding diagonal entry is

$$U_{kk}(\gamma) = \exp\left(-i\gamma\left(\sum_{i<j} J_{ij}\, s_i^{(k)} s_j^{(k)} + \sum_i h_i\, s_i^{(k)}\right)\right), \tag{20}$$

where

$$s_i^{(k)} = 1 - 2x_i^{(k)}. \tag{21}$$

Notice that we don't need to actually build the huge tensor product matrices. We just loop over all 2^n basis states, compute $s_i = 1 - 2x_i$ for each qubit, plug into the energy formula, and that gives you the diagonal of H.

Let's check the case $n = 2$ using the tensor product to verify if the unitary operator found in (20) actually works.

So, for two qubits with coupling J_{12} and local fields h_1, h_2, the Hamiltonian is

$$H = J_{12}(Z_1 Z_2) + h_1 Z_1 + h_2 Z_2. \tag{22}$$

This is a 4×4 diagonal matrix, given by

$$H = \begin{pmatrix} J_{12} + h_1 + h_2 & 0 & 0 & 0 \\ 0 & -J_{12} + h_1 - h_2 & 0 & 0 \\ 0 & 0 & -J_{12} - h_1 + h_2 & 0 \\ 0 & 0 & 0 & J_{12} - h_1 - h_2 \end{pmatrix}. \quad (23)$$

The four diagonal entries are the energies of the four computational basis states $|00\rangle, |01\rangle, |10\rangle, |11\rangle$. The ground state (optimal solution) is the basis state with the lowest energy.

Now, passing from the Hamiltonian to the unitary operator,

$$U(\gamma) = e^{-i\gamma H}. \quad (24)$$

If H has diagonal entries $[E_1, E_2, E_3, E_4]$, then

$$e^{-i\gamma H} = \begin{pmatrix} e^{-i\gamma E_1} & 0 & 0 & 0 \\ 0 & e^{-i\gamma E_2} & 0 & 0 \\ 0 & 0 & e^{-i\gamma E_3} & 0 \\ 0 & 0 & 0 & e^{-i\gamma E_4} \end{pmatrix}. \quad (25)$$

This agrees with the general n-qubit formula derived earlier. In the case $n = 2$, the diagonal entries E_1, E_2, E_3, E_4 are exactly the energies associated with the four computational basis states, so the unitary is obtained by applying the general expression to each basis state.

3 Methodology and Results

3.1 Classical Markowitz Baseline

The classical approach solves the mean-variance optimization problem exactly using convex quadratic programming. For a universe of $n = 50$ assets with expected return vector $\boldsymbol{\mu}$ and covariance matrix $\boldsymbol{\Sigma}$, we minimise

$$\min_{\mathbf{w}} \; q\,\mathbf{w}^\top \boldsymbol{\Sigma}\,\mathbf{w} - \boldsymbol{\mu}^\top \mathbf{w} \quad \text{s.t.} \quad \sum_i w_i = 1, \quad w_{i,\min} \le w_i \le w_{i,\max} \quad (26)$$

where $q \ge 0$ is a risk-aversion parameter. The problem was solved with `cvxpy` at $q = 1.0$, yielding a continuous optimal portfolio with an expected return of 8.06%, volatility of 9.10%, and a Sharpe ratio of 0.886.

To construct a directly comparable binary baseline for the quantum approach, we restricted the problem to the eight assets with the largest continuous weights and enforced

a fixed cardinality of $B = 4$ holdings, each equal-weighted at $w_i = \frac{1}{4}$. All $\binom{8}{4} = 70$ feasible portfolios were enumerated by brute force. The best portfolio at each risk-aversion level is reported in Table 1.

Table 1: Optimal binary portfolio ($B = 4$) at each risk-aversion level.

q	Return	Volatility	Sharpe	Selected Assets
0.5	8.41%	12.07%	0.697	A004, A005, A016, A030
1.0	8.21%	10.34%	0.794	A001, A004, A016, A006
2.0	8.21%	10.34%	0.794	A001, A004, A016, A006
5.0	8.06%	9.98%	0.807	A004, A005, A030, A006
10.0	8.06%	9.98%	0.807	A004, A005, A030, A006

The maximum-Sharpe binary portfolio selects assets A004, A005, A030, and A006, achieving a return of 8.06%, volatility of 9.98%, and a Sharpe ratio of 0.807. This serves as the classical combinatorial benchmark against which the quantum optimisation results are evaluated.

3.2 Pauli Propagation approach for QAOA

Simulating a quantum circuit with n qubits normally requires tracking a state vector of size 2^n. For 8 qubits, that is 256 entries, which is manageable. But when we want to scale to 20, 50, or 127 qubits, the state vector becomes impossibly large. We cannot store 2^127 complex numbers on any classical computer. Rather than tracking the full quantum state, it tracks how observables (things we want to measure) evolve through the circuit.

3.2.1 Pseudocode

1. Convert initial Pauli polynomial terms into a sparse polynomial representation.

2. Remove all terms with coefficient magnitude below ABS_CUTOFF.

3. If the number of terms exceeds max_terms, retain only the largest max_terms by coefficient magnitude.

4. For each gate/operator in the circuit:
a. Propagate each current term through the operator.
b. Merge identical resulting Pauli terms.
c. Remove terms with coefficient magnitude below ABS_CUTOFF.
d. Truncate again to max_terms if needed.

5. Return the final sparse polynomial operation set.

3.2.2 Results for Pauli Propagation Approach with noise

Table 2 shows the ten most probable bitstrings (asset selections) returned by the QAOA simulation.

Table 2: Top 10 most probable bitstrings (asset selections).

Rank	Bitstring	Assets	Selected Assets	Probability
1	$\|11001100\rangle$	4	A001, A002, A005, A006	0.0130
2	$\|00101101\rangle$	4	A003, A005, A006, A008	0.0130
3	$\|01011010\rangle$	4	A002, A004, A005, A007	0.0130
4	$\|11010010\rangle$	4	A001, A002, A004, A007	0.0130
5	$\|01110010\rangle$	4	A002, A003, A004, A007	0.0120
6	$\|01010110\rangle$	4	A002, A004, A006, A007	0.0120
7	$\|11100101\rangle$	5	A001, A002, A003, A006, A008	0.0120
8	$\|01101001\rangle$	4	A002, A003, A005, A008	0.0120
9	$\|10011010\rangle$	4	A001, A004, A005, A007	0.0120
10	$\|11011000\rangle$	4	A001, A002, A004, A005	0.0100

3.2.3 Best Portfolio Solution with noise

The optimal portfolio identified by the QAOA algorithm selects the following assets:

- **Selected assets:** A001, A002, A005, A006
- **Number of assets:** 4 (budget: 4.0)
- **Expected return:** 0.2788
- **Portfolio variance:** 0.0299
- **Portfolio risk (std dev):** 0.1729
- **Objective (return $-0.7\times$ risk):** 0.1577
- **Budget constraint satisfied** (selected 4)

3.2.4 Top 10 bitstrings: objective vs average (with noise)

Table 3: Top 10 bitstrings: objective vs average.

Rank	Bitstring	Objective	Diff vs Avg	Probability
1	$\|11001100\rangle$	−0.257862	−0.263106	0.0130
2	$\|00101101\rangle$	−0.249091	−0.254335	0.0130
3	$\|01011010\rangle$	−0.165329	−0.170573	0.0130
4	$\|11010010\rangle$	−0.172851	−0.178095	0.0130
5	$\|01110010\rangle$	−0.131469	−0.136713	0.0120
6	$\|01010110\rangle$	−0.194892	−0.200136	0.0120
7	$\|11100101\rangle$	−0.182826	−0.188070	0.0120
8	$\|01101001\rangle$	−0.219096	−0.224340	0.0120
9	$\|10011010\rangle$	−0.133620	−0.188864	0.0120
10	$\|11011000\rangle$	−0.231547	−0.236791	0.0100

3.2.5 Top 10 most probable bitstrings (asset selections) - without noise

Table 4: Top 10 most probable bitstrings (asset selections) — noisy simulation.

Rank	Bitstring	Assets	Selected Assets	Probability
1	$\|01101001\rangle$	4	A002, A003, A005, A008	0.0160
2	$\|01011001\rangle$	4	A002, A004, A005, A008	0.0130
3	$\|10101100\rangle$	4	A001, A003, A005, A006	0.0130
4	$\|00010111\rangle$	4	A004, A006, A007, A008	0.0120
5	$\|00110110\rangle$	4	A003, A004, A006, A007	0.0120
6	$\|10100110\rangle$	4	A001, A003, A006, A007	0.0120
7	$\|00011011\rangle$	4	A004, A005, A007, A008	0.0110
8	$\|00011110\rangle$	4	A004, A005, A006, A007	0.0110
9	$\|01101010\rangle$	4	A002, A003, A005, A007	0.0110
10	$\|10001101\rangle$	4	A001, A005, A006, A008	0.0110

3.2.6 Best Portfolio Solution — without noise

The optimal portfolio identified without noisy QAOA simulation selects the following assets:

- **Selected assets:** A002, A003, A005, A008

- **Number of assets:** 4 (budget: 4.0)
- **Expected return:** 0.2381
- **Portfolio variance:** 0.0271
- **Portfolio risk (std dev):** 0.1646
- **Objective (return $-0.7\times$ risk):** 0.1228
- **Budget constraint satisfied** (selected 4)

For comparison, the objective values across all $2^8 = 256$ possible bitstrings are:

- **Average objective over all 256 bitstrings:** 0.005244
- **Minimum objective over all bitstrings:** -0.302817 (bitstring $|10001101\rangle$)

3.2.7 Top 10 bitstrings: objective vs average - without noise

Table 5: Top 10 bitstrings: objective vs average — noisy simulation.

Rank	Bitstring	Objective	Diff vs Avg	Probability
1	$\|01101001\rangle$	-0.219096	-0.224340	0.0160
2	$\|01011001\rangle$	-0.229707	-0.234951	0.0130
3	$\|10101100\rangle$	-0.255711	-0.260955	0.0130
4	$\|00010111\rangle$	-0.195829	-0.201073	0.0120
5	$\|00110110\rangle$	-0.172356	-0.177600	0.0120
6	$\|10100110\rangle$	-0.180500	-0.185743	0.0120
7	$\|00011011\rangle$	-0.169264	-0.174507	0.0110
8	$\|00011110\rangle$	-0.194777	-0.200021	0.0110
9	$\|01101010\rangle$	-0.140954	-0.146198	0.0110
10	$\|10001101\rangle$	-0.302817	-0.308060	0.0110

References

[1] Markowitz mean-variance portfolio theory. https://sites.math.washington.edu/~burke/crs/408/fin-proj/mark1.pdf. Accessed: 2026-04-05.

[2] IBM Quantum Learning. Pauli operations and observables. https://quantum.cloud.ibm.com/learning/en/courses/foundations-of-quantum-error-correction/stabilizer-formalism/pauli-operations-and-observables, n.d. Accessed: 2026-04-05.

[3] Qiskit addons team. Simulating 127-qubit kicked-ising model. https://qiskit.github.io/pauli-prop/tutorials/02_simulate_127Q_kicked_ising.html, 2025. Pauli propagation 0.2.0 documentation, last updated March 16, 2026, accessed April 5, 2026.

[4] YQuantum-2026. Capgemini-The-Hartford-Challenge. https://github.com/YQuantum-2026/Capgemini-The-Hartford-Challenge, 2026. GitHub repository, accessed April 4, 2026.